Лабораторна робота №5

Реалізація алгоритму кластеризації kmeans без використання sklearn

Мета роботи: ознайомитися з принципами роботи алгоритму кластеризації та реалізувати його програмно мовою Python без використання бібліотек машинного навчання.

Теоретичні відомості

Алгоритм k-means є одним із найпоширеніших методів кластеризації, який використовується для поділу набору даних на задану кількість кластерів. Його основна ідея полягає у тому, щоб розбити об'єкти на групи таким чином, щоб об'єкти в межах одного кластера були максимально подібними між собою, а об'єкти з різних кластерів — максимально відмінними. Подібність між об'єктами зазвичай визначається за допомогою метрики відстані, найчастіше використовується евклідова відстань.

Алгоритм працює ітеративно та складається з кількох основних етапів. Спочатку випадковим чином або за допомогою спеціальних методів (наприклад, k-means++) обираються початкові центроїди кластерів. Далі кожна точка даних відноситься до того кластера, центр якого знаходиться на найменшій відстані від цієї точки. Після цього центроїди перераховуються як середнє значення всіх точок, що належать відповідному кластеру. Ці кроки повторюються доти, доки положення центроїдів не перестане суттєво змінюватися або не буде досягнуто заданої кількості ітерацій.

Однією з важливих задач при використанні алгоритму k-means є визначення оптимальної кількості кластерів. Для цього часто застосовується метод «ліктя». Його суть полягає в обчисленні суми квадратів відстаней між точками та центроїдами їхніх кластерів (SSE) для різних значень k. При збільшенні кількості кластерів значення цієї функції зменшується, але після певного моменту темп зменшення різко сповільнюється. Точка перегину графіка і називається «ліктем» та вважається оптимальним значенням k.

Алгоритм k-means добре працює для даних, які утворюють компактні та приблизно сферичні кластери. Водночас він має обмеження: чутливість до початкового вибору центроїдів, необхідність заздалегідь задавати кількість кластерів, а також складнощі при роботі з даними складної форми (наприклад, витягнутими або вкладеними структурами). Крім того, алгоритм чутливий до масштабування ознак, тому перед його застосуванням часто використовують нормалізацію або стандартизацію даних.

У випадку багатовимірних даних для їх візуалізації застосовуються методи зменшення розмірності, зокрема аналіз головних компонент (PCA), який дозволяє відобразити дані у двовимірному просторі з мінімальною втратою інформації.

Програмна реалізація

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation

# зчитуємо дані з файлу
df = pd.read_csv("IRIS.csv")

# перемішаємо дані
df = df.sample(frac=1, random_state=42)

# розділимо на ознаки та мітки
# варто зазначити, що мітки для тренування моделі не потрібні, тут
# вони тільки для візуального сприйняття, а також для перевірки точності моделі
X = df.iloc[:, [0, 1, 2, 3]]
y = df.iloc[:, 4]

##################################################################################
#
# також варто зазначити, що дані можуть мати більше ніж 2 ознаки, і часто
# саме так і буває, для цієї моделі kmeans різниці немає, оскільки метрикою є
# відстань в n-вимірному просторі, але для візуалізації ці дані потрібно
# стискати, що, безумовно, приведе до втрати інформації.
# Одним з найкращих методів є PCA() - аналіз головних компонент
#
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
X = pca.fit_transform(X)
#
##################################################################################
```

```
################################################################################
#
################################################################################
#

# це для довільних перевірок, можна створювати будь які дані
# from sklearn.datasets import make_blobs
# X, y = make_blobs(n_samples=100, n_features=2, centers=4, cluster_std=2, random_state=8)

# ВАЖЛИВО !!!!
# даний метод kmeans добре працює з даними типу blobs (хмарки),
# але погано, якщо наші дані на графіку виглядають як еліпси,
# кола, місяці, дуги, вкладені кола і інші дані зі складною геометрією

################################################################################
#
################################################################################
#

# функція для розрахунку евклідової довжини між векторами
def euclidean_distance(vector1, vector2):
    v1 = np.array(vector1)
    v2 = np.array(vector2)
    return np.sqrt(np.sum((v2 - v1) ** 2))

class KMeans:
    def __init__(self, k_max=10):
        self.__k_max = k_max # максимальна кількість класетрів для методу ліктя
        self.__rng = np.random.default_rng(seed=42)

    def train(self, X, is_find_optimal_k=False, k=None, elbow_train_animate=False):
        if not isinstance(X, np.ndarray):
            X = X.to_numpy()
        self.__X = X

        # метод ліктя для отримання кількості центроїдів
        if is_find_optimal_k == False and k == None:
            k = self.__elbow(elbow_train_animate)
        elif is_find_optimal_k == True and k is None or\
             is_find_optimal_k == False and k is not None:
            raise Exception("Error: can't be called as object's\
                            method with these parameters")


        # ініціалізація центроїдів за допомогою kmeans++
        centroids = self.__kmeans_plus_plus(k)

        # зберігаємо стани для анімації тренування моделі
        states = []

        shift = 1
```

```
epsilon = 0.01
# будемо коригувати центроїди поки зміщення не стане меншим
# за вказане значення epsilon
while shift > epsilon:
    # на кожній ітерації будемо робити наступне:

    # проходимось по усім точкам
    feat_points_idx = []
    for j in range(len(X)):

        # і для кожної точки рахуємо відстань до кожної центроїди
        point = X[j]
        dists = []
        for centroid in centroids:
            dists.append(euclidean_distance(point, centroid))

        # вибираємо найменшу відстань
        idx = np.argmin(dists)

        # додаємо мітку в загальний список
        # індекс найменшої відстані у списку - це і є мітка
        # міткою тут називається кластер, клас, область тощо
        feat_points_idx.append(idx)

    # фактично на даному етапі
    # ми отримали мітки точок при поточних центроїдах

    # зберігаємо "кадр", якщо це так можна назвати, для анімації
    states.append((X.copy(), centroids.copy(), feat_points_idx.copy()))

    # далі нам потрібно усереднити для кожного класетра точки,
    # які їм належать
    # для цього будемо зберігати словник, де ключ - кластер,
    # значення - список точок, які належать поточному кластеру
    points = {}
    feats = set(feat_points_idx)
    for feat in feats:
        points[feat] = [] # отримали щось типу {0: [], 1: [], ...}
    for i in range(len(X)):
        points[feat_points_idx[i]].append(X[i])

    # тепер будемо усереднювати точки для кожного кластера,
    # окрім цього будемо рахувати зміщення
    shift = 0
    for key, values in points.items():
         prev_centroid = centroids[key].copy()
         centroids[key] = np.mean(values, axis=0)
         shift += euclidean_distance(centroids[key], prev_centroid)

# зберігаємо стани як атрибут об'єкта - будемо малювати в окремому методі
if is_find_optimal_k == False:
    self.__states = states
```

```
    return points, centroids, feat_points_idx

# метод для ініціалізації центроїдів, щоб вони близько друг з другом не були
def __kmeans_plus_plus(self, k):
    centroids = []

    # генеруємо першу випадкову точку
    centroid_idx = self.__rng.choice(self.__X.shape[0], 1)[0]
    centroid = self.__X[centroid_idx]
    centroids.append(centroid)

    # решта центроїдів вибирається з ймовірністю, пропорційною
    # квадрату відстані до найближчого вже обраного центроїду
    while k > 1:
        min_dists = []

        # проходимся по кожній точці
        for x in self.__X:
            dists = []

            # і для кожної точки обчислюємо відстані до усіх центроїд
            # та вибираємо найменшу відстань
            for centroid in centroids:
                dists.append(euclidean_distance(x, centroid))
            min_dists.append(min(dists))

        # серед отриманих мінімальних відстаней вибираємо максимальну
        centroid_idx = np.argmax(min_dists)
        centroid = self.__X[centroid_idx]
        centroids.append(centroid)
        k -= 1

    return np.array(centroids)

# метод ліктя для визначення кількості класетрів
def __elbow(self, animate=False):
    counter = 1
    dists = []

    # суть методу в тому, що ми маємо навчати модель на декількох кластерах
    while counter <= self.__k_max:
        points, centroids, labels = self.train(self.__X, True, counter)

        # візуалізація результатів розбиття при різних k
        if animate:
            plt.figure(figsize=(8, 6))
            plt.title("Знаходження оптимального числа кластерів"\
                      f"методом ліктя (k = {counter})")
            plt.scatter(self.__X[:, 0], self.__X[:, 1], c=labels)
            plt.scatter(centroids[:, 0], centroids[:, 1],
                        color="red", marker="*", s=100)
```

```
        plt.show()

    # для кожного k ми рахуємо SSE - суму квадратів відстаней
    # між об'єктами та центроїдами кластерів
    dist = 0
    for cluster_idx, cluster_points in points.items():
        for cluster_point in cluster_points:
            dist += euclidean_distance(cluster_point,
                                       centroids[cluster_idx]) ** 2
    dists.append(dist)
    counter += 1

# візуалізуємо графік ("перегиб ліктя")
plt.title("Перегиб ліктя")
plt.plot(np.arange(1, self.__k_max + 1), dists)

# на оснві цього можемо знайти оптимальне значення k
# якщо робити це аналітично, то можна, наприклад, шукати мінімум
# фукнції частки двох похідних в наступній та поточній точках,
# тобто таким шляхом ми знайдемо найбільший перегиб графіка функції
diff = np.diff(dists)
diff_r = []
for i in range(len(diff) - 1):
    diff_r.append(diff[i+1] / diff[i])
k = np.argmin(diff_r) + 2

# окрмім цього є ще метод, в якому проводиться хорда з початкової
# точки графіка у кінцеву точку і опускаються перпендикуляри з усіх інших
# точок на цю хорду, відповідно точка з якої буде проведений найдовший
# перпендикуляр і буде наша шукана точка

# але ці аналітичні методи працюють не завжди так, як очікувалось,
# тому в реалізації цього методу залишаю можливість користувачу
# вводити кількість кластерів, дивлячись на графік

print("Метод ліктя вибирає оптимальну кількість кластерів...")
print("Аналітичний метод через мінімум функції частки похідних" \
f" дав результат: k = {k}.")
print("Тепер подивіться на графік і зробіть вибір.\nВи можете пропустити," \
" або можете ввести іншу кількість кластерів:\nоптимальним k вважається" \
" та точка, після якої SSE (сума квадратів відстаней) незначно зменшується")

plt.show()

while True:
    user_inp_k = input("Введіть ціле значення k (Enter = залишити " \
    "обчислену кількість кластерів): ")
    if user_inp_k == "":
        break
    try:
        k = int(user_inp_k)
        if k < 1:
```

```
                raise Exception("Можна вводити тільки цілі числа > 0")
            break
        except Exception as e:
            print(e)

    return k

# метод для візуалізації та оцінки результатів
def train_animate(self, y=None):
    if y is not None:
        # оскільки мітки можуть бути і рядковими даними,
        # перетворимо їх на числа, щоб
        # не було помилки у функції plt.scatter(...c=y)
        labels = []
        y_unique_list = list(set(y))
        for el in y:
            for i in range(len(y_unique_list)):
                if el == y_unique_list[i]:
                    labels.append(i)
        y = labels

    # створюємо графік з двома осями
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

    # функція оновлення кадрів
    def update(frame):
        ax1.clear()
        ax2.clear()

        X, centroids, labels = self.__states[frame]
        ax1.set_title("Training...")
        ax1.scatter(X[:, 0], X[:, 1], c=labels)
        ax1.scatter(centroids[:, 0], centroids[:, 1], c="red", marker="*", s=100)

        ax2.set_title("Real data")
        ax2.scatter(X[:, 0], X[:, 1], c=y)

    ani = FuncAnimation(fig, update, frames=len(self.__states), interval=1000)
    plt.show()

# параметр k_max вказує максимальну кількість класетрів, на яких
# буде навчатись модель для вибору оптимального числа k у методі ліктя
kmean_model = KMeans(k_max=10)

# параметр elbow_train_animate вказує чи треба візуалізувати процес
# навчання при вибору оптимального числа кластерів у методі ліктя
kmean_model.train(X, elbow_train_animate=False)

# анімація навчання у порівнянні з реальними даними (якщо вони є)
# за допомогою цього можна візуально оцінити коректну роботу моделі
kmean_model.train_animate(y)
```

кольори не відповідають дійсності якщо порівнювати два графіки,
вони просто розбивають дані на класетри

Результати експериментів

Результати зображено на рисунках 1, 2.

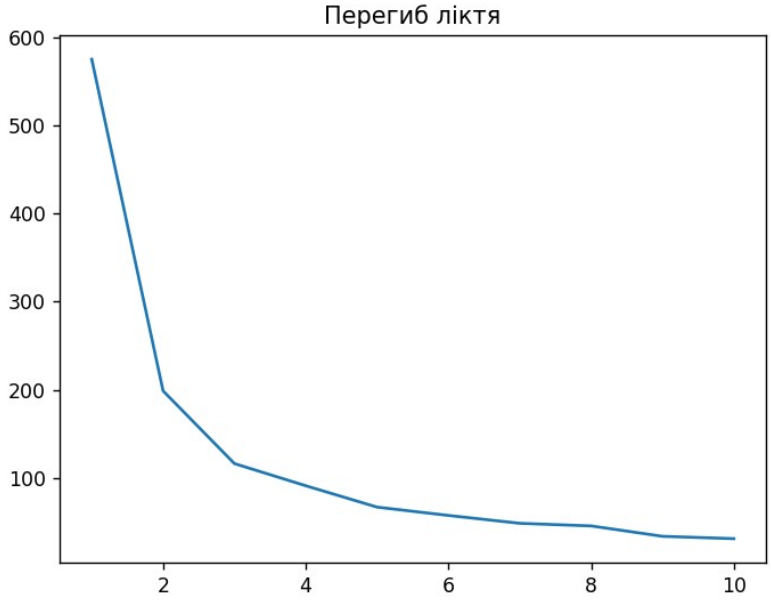

Рисунок 1 — Графік «ліктя»

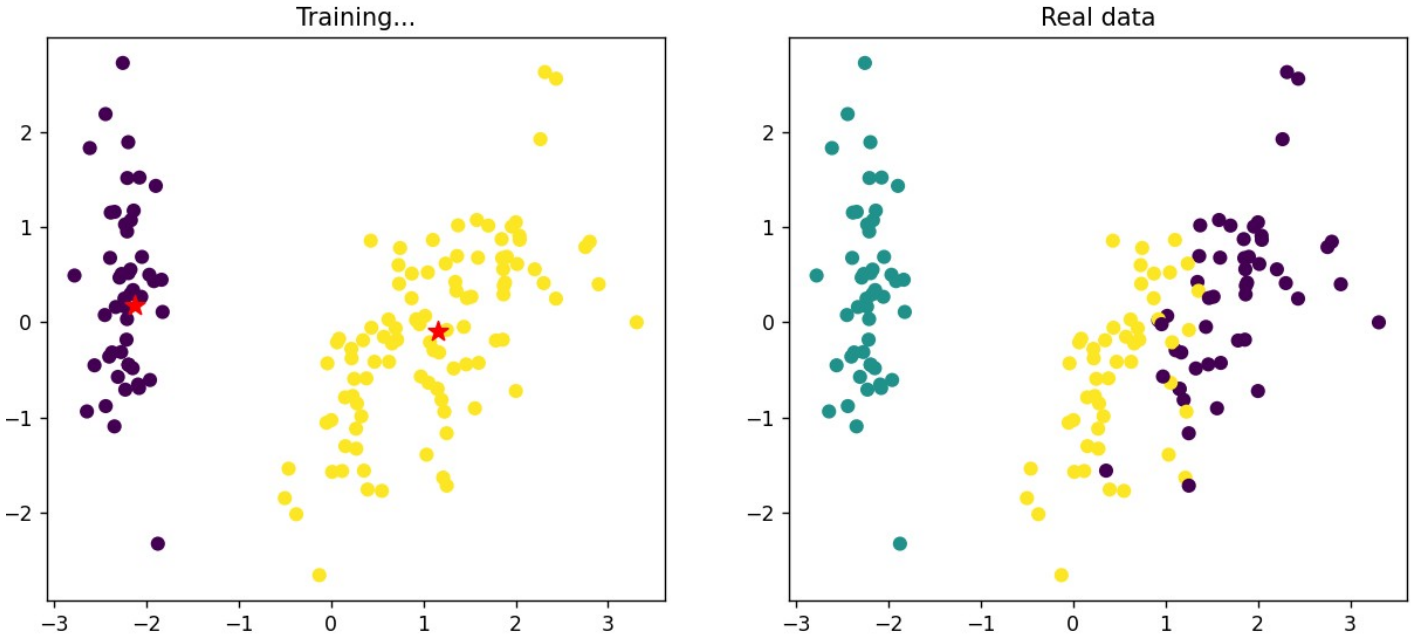

Рисунок 2 — Результат експериментів

Як бачимо аналітичний метод дав результат k = 2 для стиснених даних ірисів.

Тепер спробуємо вручну задати кількість кластерів (рисунок 3, 4).

```
Метод ліктя вибирає оптимальну кількість кластерів...
Аналітичний метод через мінімум функції частки похідних дав результат: k = 2.
Тепер подивіться на графік і зробіть вибір.
Ви можете пропустити, або можете ввести іншу кількість кластерів:
оптимальним k вважається та точка, після якої SSE (сума квадратів відстаней) незначно зменшується
Введіть ціле значення k (Enter = залишити обчислену кількість кластерів): 3
```

Рисунок 3 — Вводимо кількість кластерів

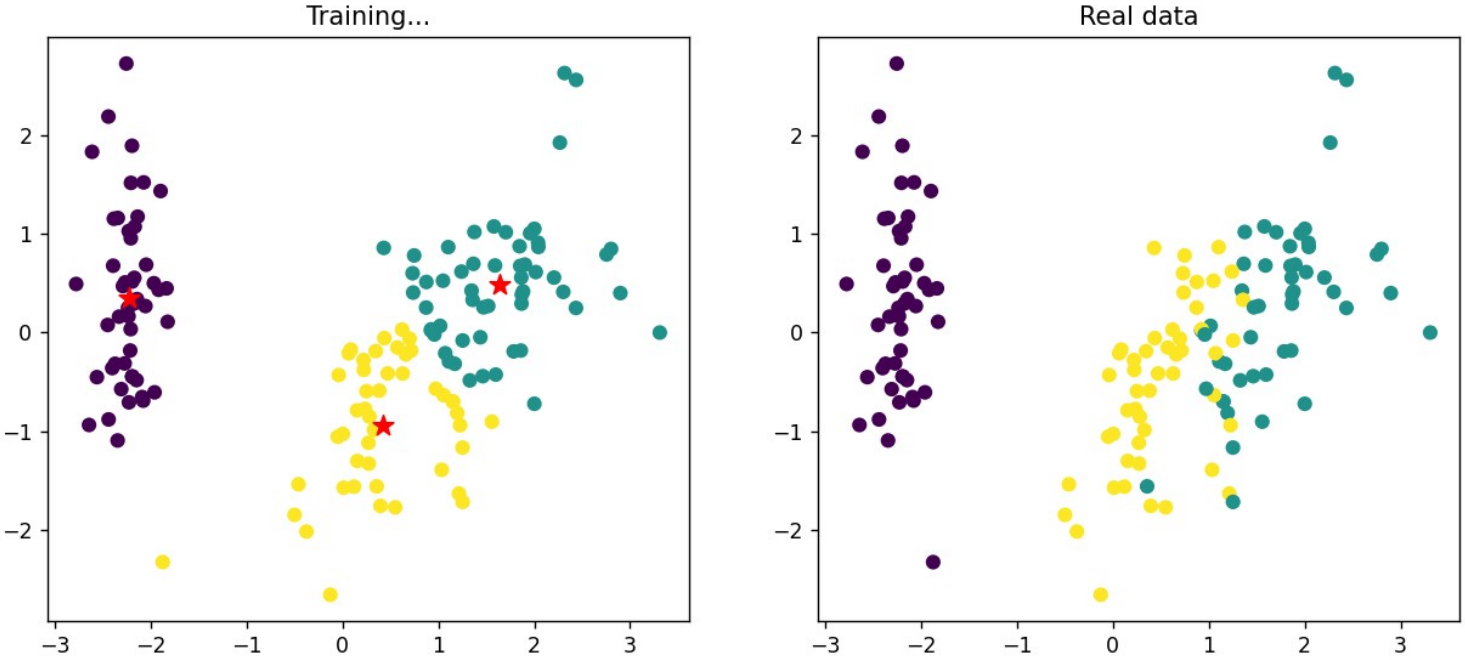

Рисунок 4 — Кластеризація стиснених ірисів при k = 3

Тепер спробуємо створити власні дані і протестувати їх (рисунок 5, 6).

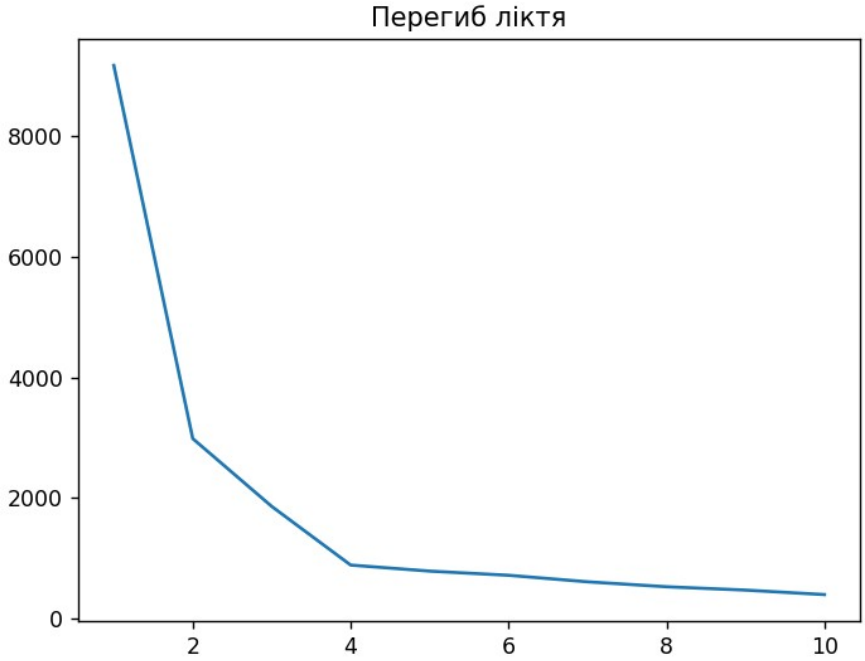

Рисунок 5 — Графік SSE для інших даних

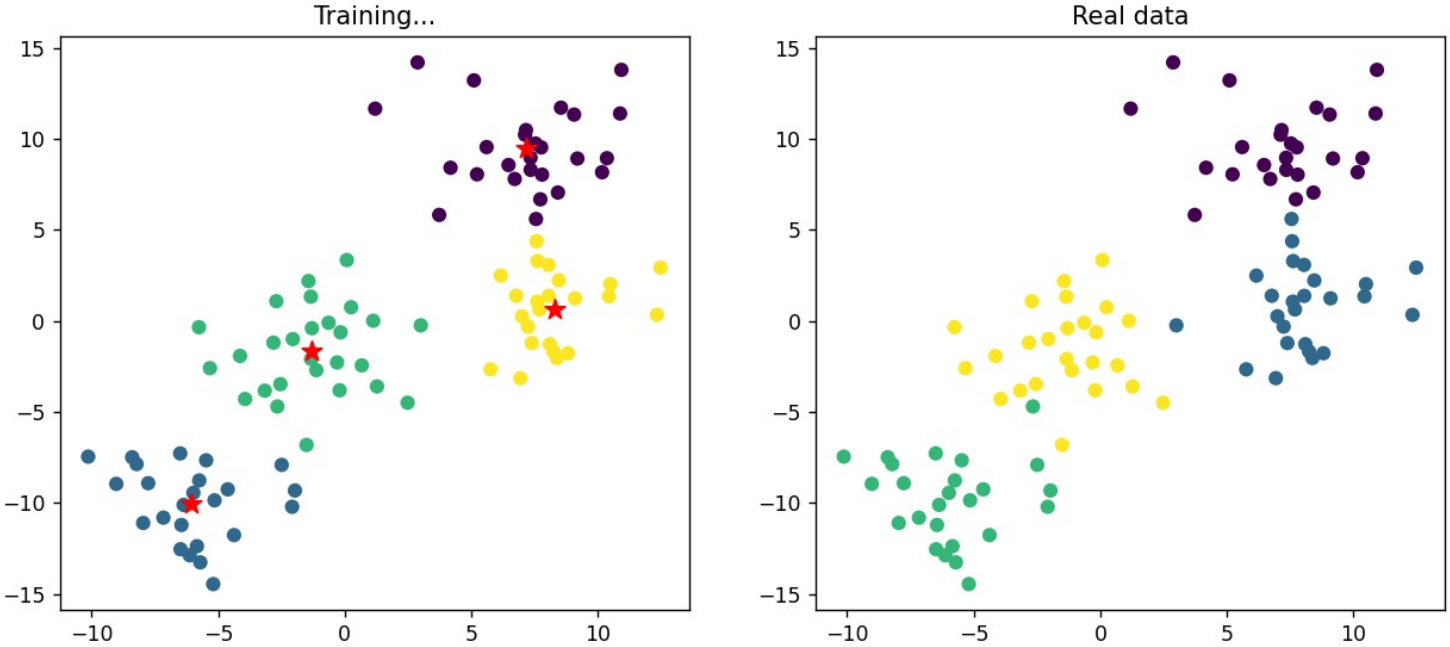

Рисунок 6 — Результат для іншого згенерованого датасету

Висновок

У ході виконання лабораторної роботи було розглянуто принципи роботи алгоритму кластеризації k-means та реалізовано його програмно мовою Python без використання спеціалізованих бібліотек машинного навчання. Було реалізовано основні етапи алгоритму, зокрема ініціалізацію центроїдів, віднесення точок до найближчих кластерів, перерахунок центрів кластерів та ітераційний процес до збіжності. Також було реалізовано метод k-means++ для покращення початкового вибору центроїдів і метод «ліктя» для визначення оптимальної кількості кластерів.

У процесі експериментів було продемонстровано роботу алгоритму на тестових даних, а також побудовано графік методу «ліктя», що дозволило оцінити оптимальне значення кількості кластерів. Додатково було реалізовано візуалізацію процесу навчання у вигляді анімації, що дозволило наочно спостерігати зміну положення центроїдів та формування кластерів. Також стало зрозуміло, що в таких типах задачах аналітичні методи не завжди дають очікуваний результат.

Джерела:

1. Метод локтя для модели KMeans с нуля | К-средних | Elbow Method | KMeans часть 4 | МАШИННОЕ ОБУЧЕНИЕ
2. Implementation of Principal Component Analysis(PCA) in K Means Clustering | by Wamika Jha | Analytics Vidhya | Medium